C# | ASP.NET Core | Web API | Entity Framework Core

120+ Interview Questions and Answers

Prepared By: Kanaiya Katarmal

Role : CTO | Software Architect

Description:

This document contains a comprehensive collection of 120+ interview questions and answers covering:

1. C# Fundamentals (1–18)
2. ASP.NET Core (19–35)
3. Web API (36–53)
4. Entity Framework Core (54–68)
5. SQL Server (69–88)
6. Performance Optimization (89–104)
7. Security (105–116)
8. Testing (117–123)
9. Architecture (124–131)

Designed to help developers prepare for technical interviews and strengthen their understanding of modern .NET technologies.

Purpose:

To provide a structured and practical guide for interview preparation, quick revision, and concept clarity in .NET development.

Linkedin : https://www.linkedin.com/in/kanaiyakatarmal
Email Id : katarmal.kanaiya@gmail.com
Newsletter : https://newsletter.kanaiyakatarmal.com

1. C# Fundamentals (1–18)

1. What is C#?
C# is a modern, object-oriented programming language developed by Microsoft as part of the .NET platform. It is widely used for building web applications, APIs, desktop apps, and cloud-based solutions. It provides strong typing, memory management, and rich libraries.

2. What are OOP principles?
OOP includes encapsulation, inheritance, polymorphism, and abstraction. These principles help developers design modular, reusable, and maintainable code. They are fundamental for building scalable enterprise applications.

3. What is encapsulation?
Encapsulation is the concept of hiding internal data and exposing only necessary functionality using properties or methods. It helps protect data from unauthorized access and ensures better control over how data is modified.

4. What is inheritance?
Inheritance allows one class to reuse the properties and methods of another class. It promotes code reusability and reduces duplication. It also helps in building hierarchical relationships between classes.

5. What is polymorphism?
Polymorphism allows methods to behave differently based on the context. It can be achieved through method overloading (compile-time) and method overriding (runtime). This improves flexibility in application design.

6. What is abstraction?
Abstraction hides complex implementation details and shows only essential features. It is achieved using abstract classes and interfaces. It helps simplify code and focus on what an object does rather than how it works.

7. What is an interface?
An interface defines a contract that a class must implement. It contains method declarations without implementation. It helps achieve abstraction and supports multiple inheritance in C#.

8. Abstract class vs Interface?
An abstract class can have both implemented and abstract methods, while an interface mainly contains method definitions. Interfaces are more flexible for multiple inheritance scenarios.

9. What is string immutability?
In C#, strings are immutable, meaning once created, they cannot be changed. Any modification creates a new string object. This ensures thread safety but may impact performance if used excessively.

10. String vs StringBuilder?
String is immutable and suitable for fixed text, while StringBuilder is mutable and efficient for frequent string modifications. It improves performance in loops or large text processing.

11. What is exception handling?
Exception handling is used to manage runtime errors using try-catch-finally blocks. It prevents application crashes and ensures graceful error handling and logging.

12. What is garbage collection?
Garbage collection is an automatic memory management feature that removes unused objects from memory. It helps prevent memory leaks and improves application performance.

13. What is delegate?
A delegate is a type-safe function pointer that holds references to methods. It is commonly used in event handling and callback mechanisms.

14. What is event?
An event is a mechanism that allows communication between objects. It is built on delegates and is used in scenarios like UI interactions or notifications.

15. What is LINQ?
LINQ (Language Integrated Query) allows querying data from collections, databases, or XML using a consistent syntax. It improves readability and reduces the need for complex loops.

16. Difference between class vs record vs struct
A class is a reference type used for complex objects. A struct is a value type stored on the stack and is lightweight. A record is a reference type designed for immutable data and supports value-based equality.

17. Explain Nullable Reference Types
Nullable reference types help avoid null reference exceptions by explicitly marking variables as nullable or non-nullable. It enables compile-time checks and improves code safety.

18. Explain Generics
Generics allow creating reusable and type-safe code components like classes and methods. They reduce code duplication and eliminate the need for type casting.

---

2. .NET Core (19–35)

19. What is .NET Core?
.NET Core is a cross-platform, open-source framework used to build modern, scalable, and high-performance applications. It supports Windows, Linux, and macOS, making it ideal for cloud-based systems.

20. What is CLR?
The Common Language Runtime (CLR) is responsible for executing .NET applications. It manages memory, handles exceptions, enforces security, and performs garbage collection.

21. What is asynchronous programming?
Asynchronous programming allows tasks to run without blocking the main thread using async and await. It improves application responsiveness and scalability, especially in web applications.

22. Sync vs Async?
Synchronous execution blocks the thread until the task completes, while asynchronous execution allows other operations to continue. Async improves performance in I/O-bound operations.

23. What is Task?
A Task represents an asynchronous operation in .NET. It is used with async/await to manage background work efficiently without blocking the main thread.

24. What is dependency injection?
Dependency Injection is a design pattern used to achieve loose coupling between classes. Dependencies are provided externally, improving testability and maintainability.

25. What are service lifetimes?
Service lifetimes define how services are created: Transient (new every time), Scoped (per request), and Singleton (one instance for the entire app).

26. What is middleware?
Middleware are components that handle HTTP requests and responses in a pipeline. Each middleware can process, modify, or pass the request to the next component.

27. What is logging?
Logging is used to record application events, errors, and performance data. It helps in debugging and monitoring applications in production environments.

28. What is configuration?
Configuration in .NET Core allows managing application settings using files like appsettings.json, environment variables, or secret managers.

29. What is Kestrel?
Kestrel is a cross-platform web server used by ASP.NET Core applications. It is lightweight, fast, and commonly used behind reverse proxies.

30. What is hosting?
Hosting refers to the environment where the application runs. ASP.NET Core supports in-process and out-of-process hosting models.

31. What is environment?
Environments like Development, Staging, and Production allow different configurations for different stages of deployment.

32. What is DI container?
The DI container is responsible for creating and managing object lifetimes. It automatically resolves dependencies in the application.

33. What are filters?
Filters are used to run code before or after controller actions. They are useful for logging, authorization, and exception handling.

34. Name 3 ways to create middleware
Middleware can be created using a class with Invoke/InvokeAsync, inline middleware using app.Use(), or extension methods. These approaches help structure request handling in a reusable way.

35. What is Background Service?
Background services are used to run long-running tasks outside HTTP requests. They are implemented using IHostedService or BackgroundService and are useful for jobs like email processing or queue handling.

---

3. Web API (36–53)

36. What is Web API?
ASP.NET Core Web API is used to build HTTP-based services that can be consumed by various clients. It follows REST principles and typically uses JSON for communication. It is widely used in modern backend systems.

37. What is REST?
REST is an architectural style that uses standard HTTP methods like GET, POST, PUT, and DELETE. It is stateless, meaning each request contains all required information.

38. What is routing?
Routing maps incoming HTTP requests to specific controller actions. It helps determine which logic should handle a request.

39. What is attribute routing?
Attribute routing allows defining routes directly on controllers and actions. It provides better flexibility and readability.

40. What is IActionResult?
IActionResult allows returning different HTTP responses like OK, BadRequest, or NotFound. It helps standardize API responses.

41. What is model binding?
Model binding automatically maps request data to action parameters. It reduces manual parsing of inputs.

42. What is model validation?
Validation ensures incoming data meets defined rules using attributes. It prevents invalid data from entering the system.

43. What is Swagger?
Swagger is used to document and test APIs. It provides an interactive UI for exploring endpoints.

44. What is CORS?
CORS controls how resources are shared across domains. It enhances security while allowing controlled access.

45. What is API versioning?
API versioning allows multiple versions of an API to coexist. It ensures backward compatibility.

46. What is HttpClient?
HttpClient is used to send HTTP requests to external APIs. It supports async operations.

47. What is throttling?
Throttling limits the number of API requests to prevent server overload and abuse.

48. What is middleware pipeline?
It is the sequence of components that process requests and responses in ASP.NET Core.

49. What is serialization?
Serialization converts objects into JSON or XML format for data transfer.

50. What are HTTP status codes?
Status codes indicate the result of an HTTP request, such as 200 (OK), 404 (Not Found), or 500 (Server Error).

51. What is Rate Limiting and its types?
Rate limiting controls the number of requests a client can make within a time period. It prevents abuse and improves stability. Common types include Fixed Window, Sliding Window, and Token Bucket.

52. Explain [FromBody], [FromQuery], [FromRoute], [FromForm]
These attributes specify where data comes from in an HTTP request. They help control model binding clearly and improve API readability.

53. What is Method Injection in Controller?
Method injection allows dependencies to be injected directly into action methods. It is

useful when a dependency is needed only in specific actions instead of the entire controller.

---

4. Entity Framework Core (54–68)

54. What is EF Core?
Entity Framework Core is an Object-Relational Mapper (ORM) that allows developers to interact with databases using C# objects instead of writing raw SQL queries. It simplifies data access and improves productivity. It supports multiple databases like SQL Server, MySQL, and PostgreSQL.

55. What is DbContext?
DbContext is the primary class in EF Core that manages database connections and tracks changes made to entities. It acts as a bridge between the application and the database. It is responsible for querying and saving data.

56. What is DbSet?
DbSet represents a table in the database and is used to query and save instances of entities. It allows performing CRUD operations using LINQ.

57. What is migration?
Migrations are used to update the database schema when the data model changes. They help maintain synchronization between code and database structure.

58. What is LINQ in EF?
LINQ is used to query data in EF Core using C# syntax. It allows writing readable and type-safe queries instead of raw SQL.

59. What is lazy loading?
Lazy loading loads related data only when it is accessed. It reduces initial query load but may result in multiple database calls.

60. What is eager loading?
Eager loading retrieves related data along with the main entity using methods like Include. It reduces the number of queries but may fetch extra data.

61. What is explicit loading?
Explicit loading is when related data is loaded manually using separate queries. It provides more control over when data is fetched.

62. What is change tracking?
Change tracking monitors modifications made to entities so EF Core knows what needs to be updated in the database. It ensures efficient updates.

63. What is AsNoTracking?
AsNoTracking disables change tracking for read-only queries, improving performance when updates are not required.

64. What is repository pattern?
The repository pattern abstracts data access logic from business logic. It improves code maintainability and testability.

65. What is Unit of Work?
Unit of Work manages multiple database operations as a single transaction. It ensures consistency and reduces database calls.

66. What is navigation property?
Navigation properties define relationships between entities, such as one-to-many or many-to-many relationships.

67. What is shadow property?
Shadow properties are properties not defined in the entity class but maintained by EF Core internally, often used for tracking purposes.

68. What is concurrency handling?
Concurrency handling ensures that multiple users do not overwrite each other's data. It is managed using techniques like row versioning or timestamps.

---

5. SQL Server (69–88)

69. What is SQL Server?
SQL Server is a relational database management system used to store, manage, and retrieve data. It supports structured data and complex queries.

70. What is normalization?
Normalization is the process of organizing data into multiple tables to reduce redundancy and improve data integrity.

71. What is denormalization?
Denormalization combines tables to improve read performance. It is often used in reporting or high-performance systems.

72. What is indexing?
Indexing improves the speed of data retrieval by creating a data structure that allows quick lookups, similar to an index in a book.

73. What is clustered index?
A clustered index determines the physical order of data in a table. Each table can have only one clustered index.

74. What is non-clustered index?
A non-clustered index stores data separately and contains pointers to the actual data rows. A table can have multiple non-clustered indexes.

75. What is primary key?
A primary key uniquely identifies each record in a table and ensures data integrity.

76. What is foreign key?
A foreign key establishes a relationship between two tables, ensuring referential integrity.

77. What is a transaction?
A transaction is a group of operations executed as a single unit. It ensures data consistency.

78. What are ACID properties?
ACID stands for Atomicity, Consistency, Isolation, and Durability. These properties ensure reliable database transactions.

79. What are isolation levels?
Isolation levels control how transactions interact with each other, preventing issues like dirty reads and phantom reads.

80. What is deadlock?
A deadlock occurs when two processes wait for each other to release resources, causing both to stop indefinitely.

81. How to prevent deadlocks?
Deadlocks can be prevented by proper indexing, maintaining consistent query order, and keeping transactions short.

82. What is stored procedure?
A stored procedure is a precompiled SQL query that can be reused. It improves performance and security.

83. What is a view?
A view is a virtual table created from a query. It simplifies complex queries and enhances security.

84. What is join?
Joins are used to combine data from multiple tables based on related columns.

85. What is pagination?
Pagination retrieves data in smaller chunks, improving performance and user experience.

86. What is query optimization?
Query optimization improves performance by rewriting queries, using indexes, and analyzing execution plans.

87. What is SQL injection?
SQL injection is a security vulnerability where malicious SQL is inserted into queries.

88. How to prevent SQL injection?
Use parameterized queries, ORM tools, and input validation to prevent SQL injection.

6. Performance Optimization (89–104)

89. How to improve API performance?
API performance can be improved using caching, asynchronous programming, efficient database queries, and proper indexing. Reducing unnecessary data transfer also helps.

90. What is caching?
Caching stores frequently accessed data in memory to reduce database calls and improve response time.

91. Types of caching?
Common types include in-memory caching, distributed caching (like Redis), and response caching.

92. What is connection pooling?
Connection pooling reuses database connections instead of creating new ones, improving performance and reducing overhead.

93. What is pagination?
Pagination limits the amount of data returned in each request, improving performance and reducing load.

94. What is compression?
Compression reduces the size of API responses, improving network performance and load times.

95. What is load balancing?
Load balancing distributes incoming traffic across multiple servers to ensure reliability and performance.

96. What is CDN?
A Content Delivery Network delivers content from servers closer to users, improving speed and reducing latency.

97. What is profiling?
Profiling analyzes application performance to identify bottlenecks and optimize resource usage.

98. What is async scalability?
Async scalability allows handling more users by freeing threads during I/O operations.

99. What is thread pool?
A thread pool manages reusable threads, reducing the cost of creating new threads for each request.

100. What is batching?
Batching groups multiple operations into a single request to reduce overhead.

101. Lazy loading performance impact?
Lazy loading reduces initial load but may cause multiple database queries.

102. Eager loading performance impact?
Eager loading reduces queries but may load unnecessary data.

103. What is caching strategy?
Caching strategy defines what data to cache, when to cache, and when to invalidate it.

104. Difference between IMemoryCache and IDistributedCache
IMemoryCache stores data in local memory and is fast but limited to a single server. IDistributedCache stores data in external systems like Redis, allowing sharing across multiple servers in distributed environments.

---

7. Security (105–116)

105. What is authentication?
Authentication verifies the identity of a user using credentials like username and password.

106. What is authorization?
Authorization determines what resources a user is allowed to access after authentication.

107. What is JWT?
JWT (JSON Web Token) is a compact token used for secure authentication and data exchange between client and server.

108. What is OAuth?
OAuth is a protocol that allows third-party applications to access user data without sharing credentials.

109. What is HTTPS?
HTTPS is a secure communication protocol that encrypts data between client and server.

110. What is hashing?
Hashing converts data into a fixed-length string, commonly used for storing passwords securely.

111. What is salting?
Salting adds random data to passwords before hashing to prevent attacks like rainbow table attacks.

112. What is role-based authorization?
Access is granted based on user roles like Admin or User.

113. What is claims-based authorization?
Authorization is based on user claims such as permissions or identity attributes.

114. API security best practices?
Use HTTPS, validate inputs, implement authentication, and avoid exposing sensitive data.

115. Explain how JWT tokens work
JWT tokens are used for authentication and contain encoded user information. After login, the server generates a token that the client sends with each request for validation.

116. Explain Refresh Tokens
Refresh tokens are long-lived tokens used to generate new access tokens without requiring re-login. They improve security by keeping access tokens short-lived.

---

8. Testing (117–123)

117. What is unit testing?
Unit testing involves testing individual components or methods in isolation to ensure they work correctly.

118. What is integration testing?
Integration testing verifies how different components work together as a system.

119. What is mocking?
Mocking simulates dependencies in tests to isolate the unit being tested.

120. What is xUnit?
xUnit is a popular testing framework used in .NET applications.

121. What is test coverage?
Test coverage measures how much of the code is tested by automated tests.

122. What is TDD?
Test-Driven Development involves writing tests before writing actual code.

123. What is automation testing?
Automation testing uses tools to run tests automatically, improving efficiency.

---

9. Architecture (124–131)

124. What is microservices?
Microservices architecture breaks applications into small, independent services that communicate via APIs. It improves scalability and flexibility.

125. What is monolithic architecture?
A monolithic architecture is a single unified application where all components are tightly coupled.

126. What are SOLID principles?
SOLID principles are design guidelines that improve code maintainability and scalability.

127. What is clean architecture?
Clean architecture separates concerns into layers, making applications easier to maintain and test.

128. What is CQRS?
CQRS separates read and write operations to improve performance and scalability.

129. What is Docker?
Docker is a platform that packages applications into containers for consistent deployment.

130. What is Kubernetes?
Kubernetes manages and orchestrates containerized applications at scale.

131. What is CI/CD?
CI/CD automates the process of building, testing, and deploying applications.